

Datadog

1. What is Datadog?

Datadog is a hosted monitoring and observability platform. It collects metrics, logs and traces from an application and the machines it runs on, and shows them in one place.

It is a service, not something we install and operate. An agent runs on our side and sends data to Datadog, which stores it, graphs it and alerts on it.

2. What problem does Datadog solve?

It removes the work of correlating separate tools during an incident.

Without one platform	With one platform
Metrics in one tool, logs in another, traces in a third	One place, one set of tags
Matching timestamps by hand across tabs	Click a spike, see the traces behind it
Each tool has its own query language	One query language

The value is the join, not the storage. Any of these signals can be stored cheaply somewhere else. What is hard, and what Datadog sells, is going from a graph to the exact requests that made it and then to their logs without leaving the page.

3. Observability vs monitoring

Monitoring tells us that something is wrong; Observability helps us understand why it is wrong.

Monitoring	Observability
Watches failure modes we predicted	Answers questions we did not predict
"Is CPU above 90 percent?"	"Why is checkout slow for one customer?"
Set up in advance	Explored after the fact
Dashboards and alerts	Dashboards, alerts, and the raw data behind them

Monitoring is part of observability, not the opposite of it. A system is observable when the data already being collected can answer a question nobody thought to ask, which is what makes the difference on a new kind of failure.

4. Metrics

Metrics are numbers that show the performance and health of a system.

Type	Measures	Example
Count	How many in an interval	Charges created
Gauge	A value at a moment	Queue depth, heap used
Rate	Count divided by time	Requests per second
Distribution	The spread, for percentiles	Request latency at p99

```
StatsDClient statsd = new NonBlockingStatsDClientBuilder()
    .prefix("payroll")
    .hostname("localhost")
    .port(8125)
    .build();

statsd.incrementCounter("charges.created");
statsd.gauge("queue.depth", queue.size());
```

Metrics are aggregated, so they are cheap and they lose detail. They answer how many and how fast. They cannot answer what happened to one particular request.

5. Logs

Logs are records of events and activities generated by an application or system.

```
{"timestamp":"2026-09-16T10:31:02Z","level":"ERROR","service":"payroll",
  "env":"production","message":"charge failed","order_id":41,
  "dd.trace_id":"7c9e2a1f5b3d"}
```

Write logs as JSON, not as sentences. A structured log can be filtered by `order_id` or grouped by `level`. A line of prose can only be searched for text.

The `dd.trace_id` field is what links a log to its trace. With it, one click moves between the request's timing and its log lines.

Logs answer what happened to one request, and they are the most expensive signal to store.

6. Traces

Traces show the complete journey of a request as it moves through different services in an application.

Each step in that journey is a span, with a start time and a duration.

```
payroll POST /api/v1/charges 412 ms
├─ postgres SELECT employees 18 ms
├─ payment-service POST /charges 380 ms
│   └─ postgres INSERT charges 22 ms
```

Traces answer where the time went. The example shows 380 of 412 milliseconds spent in one downstream call, which no metric and no log would have told us as directly.

Instrumenting a Java service needs no code changes; the tracer is attached to the JVM:

```
java -javaagent:dd-java-agent.jar -jar payroll.jar
```

7. Events

Datadog Events are records of important changes or activities in our infrastructure and applications: a deploy, a configuration change, a monitor firing.

Events are markers, not measurements. Their value is overlaying them on a graph, so a change in a metric lines up with the deploy that caused it.

This is the deployment marker from a CI/CD pipeline. Sending an event at the end of a deploy turns "errors rose at 10:31" into "errors rose when we released 1.4.2".

8. Dashboards

A dashboard is a saved set of graphs shown together.

Kind	For
Timeboard	Investigation. Every graph shares one time range and a cursor
Screenboard	Status displays. Free layout, mixed time ranges

A dashboard should answer one question. A service dashboard showing error rate, latency, throughput and saturation is useful. One with forty graphs is a place people go to feel informed rather than to find something.

9. Monitors

Monitors are rules in Datadog that continuously check our data and trigger an alert when a defined condition is met.

```
avg(last_5m):avg:payroll.request.errors{env:production} > 25
```

State	Means
OK	The condition is not met
Warn	Past the warning threshold, not the alert one
Alert	Past the alert threshold
No Data	Nothing reported in the window

No Data is a real state and it matters. A service that stopped sending metrics looks healthy to a threshold check, so a monitor that does not treat missing data as a problem will stay green through an outage.

The type of a monitor is decided by what it watches:

Monitor	Watches	Use it for
Metric	A metric query	Error counts, queue depth, CPU. The most common
APM	Trace data per service or endpoint	Latency at p99, error rate on one endpoint
Log	How many logs match a query	A spike in <code>level:ERROR</code> , or a specific message
Event	Events matching a query	A deploy happening, a host being terminated
Composite	Other monitors, combined	Reducing noise

A composite monitor is how two weak signals become one useful alert. Either signal alone is noisy; together they mean something.

```
a && b
  a = error rate above 5%
  b = traffic above 100 requests per minute
```

Without `b`, three failures at four in the morning against six requests would page somebody. With it, the alert only fires when enough traffic is arriving for the error rate to mean anything.

Prefer an APM monitor to a metric monitor for latency. APM already breaks latency down by endpoint, so the alert says which route is slow rather than that the service is.

10. Alerts

Alerts are notifications sent when a defined condition or threshold is met.

Monitor and alert are not the same thing. The monitor is the rule; the alert is the message it produces. The words get used for each other constantly.

```
@slack-payroll-oncall @pagerduty-payroll
Error rate is {{value}} over the last 5 minutes.
Runbook: https://wiki.company.com/payroll/errors
```

Two rules keep alerts worth reading:

Alert on symptoms, not causes. Alert when users are getting errors, not when CPU is high. High CPU that nobody notices is not an incident, and an outage with normal CPU still is.

Every alert needs an action. If the response is to look at it and close it, it should be a dashboard, not a page. Alerts nobody acts on train people to ignore the ones that matter.

11. What is the Datadog Agent, and why is it required?

Datadog Agent is a lightweight software that runs on our machine or server and collects metrics, logs, and other monitoring data and sends them to Datadog.

Important point: We don't install Datadog itself on every server. We install the Datadog Agent, which collects the data and sends it to Datadog's hosted platform.

It collects	From
System metrics	The host: CPU, memory, disk, network
Integration metrics	Services on the host, such as Postgres
Logs	Files, or the container's output
Traces	The tracer library inside the application

It is required because Datadog is outside our network. Something on our side has to gather the data and send it, and doing that in each application instead would mean four problems:

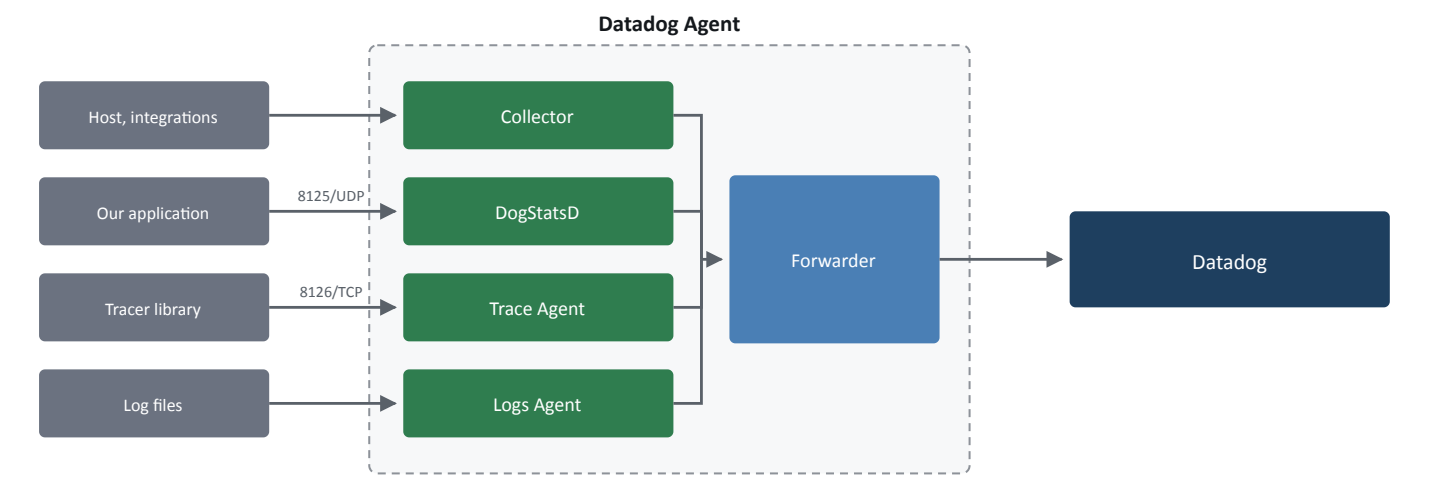
Without an Agent	With the Agent
Every service needs an API key	One key, held by the Agent
Every service opens its own connection	One batched, compressed stream
A network failure loses data	The Agent buffers and retries
Nothing reports host CPU or disk	The Agent sees the whole machine

The Agent collects; the tracer instruments. Metrics and logs reach Datadog through the Agent without the application knowing. Traces need the tracer library attached to the application itself.

Where it runs	How
A virtual machine	Installed as a service
Kubernetes	A DaemonSet, so one Agent per node
A container host	Its own container

12. Agent architecture

The Agent is one process containing several parts, each responsible for one kind of data.



Four parts collect and one sends. Everything leaves through the Forwarder.

Part	Does
Collector	Runs system and integration checks on a schedule
Forwarder	Batches everything and sends it, retrying on failure
DogStatsD	Listens for custom metrics from our applications
Trace Agent	Receives spans from tracer libraries
Logs Agent	Tails log files, or reads the container's output

Two of those listen on ports, and both are worth remembering:

8125/UDP

DogStatsD, for custom metrics

8126/TCP

Trace Agent, for spans

The Forwarder is why a network problem does not lose data. Checks keep running and results queue on disk, so a short outage between us and Datadog produces a gap of seconds rather than of minutes.

13. What is DogStatsD?

DogStatsD is a Datadog service that collects custom metrics from applications and sends them to Datadog.

It runs inside the Agent, which is what the metrics are sent to before the Forwarder passes them on.

It is Datadog's version of StatsD, with tags and percentile distributions added.

```
statsd.incrementCounter("charges.created", "env:production", "tier:premium");
```

It uses UDP, on purpose. The call returns immediately and never waits for the Agent, so metric collection cannot slow down or block a request.

The trade is that delivery is not guaranteed. A dropped packet is a lost data point, which is acceptable for a counter sampled thousands of times and wrong for anything that must not be lost. Nothing that needs to be exact should go through DogStatsD.

14. Datadog integrations

Datadog integrations are connections that allow Datadog to collect and monitor data from external services, applications, and infrastructure.

Each one is prebuilt configuration for a specific technology, enabled on the Agent.

```
init_config:
instances:
  - host: db.internal
    port: 5432
    username: datadog
```

Enabling one brings three things at once: the metrics that technology exposes, a dashboard already built for them, and suggested monitors.

Look for an integration before writing a custom metric. Datadog ships several hundred, covering Postgres, Kafka, Redis, the JVM, Kubernetes and the cloud providers. A custom metric that duplicates one is work nobody needed to do.

15. Tags

Tags are labels added to data to identify, group, and filter it easily. They are written as a key and a value, such as `env:production` or `service:payroll`.

Tags are what make one query cross the signals. The same `service:payroll env:production` selects the metrics, the logs and the traces, which is what turns three datasets into one investigation.

Three tags are reserved and matter more than the rest:

Tag	Means
env	Which environment: <code>production</code> , <code>staging</code>
service	Which service produced this
version	Which release produced this

```
java -javaagent:dd-java-agent.jar \  
  -Ddd.env=production -Ddd.service=payroll -Ddd.version=1.4.2 \  
  -jar payroll.jar
```

Setting all three is called unified service tagging, and it is what lets Datadog compare one release against the previous one.

Inconsistent tags are the most common thing to get wrong. A service tagged `payroll` in one place and `payroll-service` in another produces two sets of data that no query joins, and the cost of fixing it grows with every dashboard and monitor built on the wrong one.

16. Application Performance Monitoring (APM)

Application Performance Monitoring (APM) is a way to monitor an application's performance and find problems such as slow requests, errors, and failures.

In Datadog it is built on traces: they are the data, and APM is what it makes from them.

It gives	Which answers
Service map	What calls what
Latency per endpoint	Which endpoint is slow, not just which service
Error rate per endpoint	Where the failures are
Slowest traces	A real example to open
Database queries	Which SQL, and how long it took

Four words appear throughout, and mixing them up makes the screens hard to read:

Term	Means
Service	One application, such as <code>payroll</code>
Resource	One endpoint or query, such as <code>POST /api/v1/charges</code>
Trace	One request's whole journey
Span	One step inside a trace

Metrics say a service is slow; APM says which endpoint and which call. That is the difference worth stating, and it is why both exist.

17. Datadog Java Agent

The Java Agent is a jar attached to the JVM that instruments the application as classes load, with no code change.

```
java -javaagent:dd-java-agent.jar \
  -Ddd.env=production -Ddd.service=payroll -Ddd.version=1.4.2 \
  -jar payroll.jar
```

It recognizes the common libraries on its own and creates spans for them:

Instrumented automatically	Produces
Servlets, Spring MVC	A span per incoming request
JDBC	A span per query, with the SQL
HTTP clients	A span per outgoing call
Kafka, JMS	Spans that link producer to consumer

Those spans go to the Trace Agent on port 8126, which is why the Agent has to be reachable from the application.

No code change is the point. Everything is a JVM flag or an environment variable, so tracing can be turned on for an existing service without touching it.

When a specific method matters, one annotation adds a span:

```
@Trace(operationName = "payroll.calculate")
public BigDecimal calculate(Employee employee) { ... }
```

18. What is profiling?

Profiling measures what the code itself is doing while it runs: which methods use the CPU, which allocate memory, and where threads are waiting.

APM says which endpoint is slow. Profiling says which method. That is the whole relationship between the two, and it is why both are needed.

```
java -javaagent:dd-java-agent.jar \
  -Ddd.profiling.enabled=true \
  -jar payroll.jar
```

Kind	Measures	Finds
CPU	Time methods spend on a processor	A hot loop, an expensive parse
Wall-clock	Elapsed time, including waiting	A slow query, a lock, an idle thread
Allocation	What creates objects, however briefly	Garbage collection pressure
Memory (heap)	What is still holding memory	A leak
Thread and lock	Where threads block on each other	Contention

CPU and wall-clock are the pair that gets confused. A thread waiting on a database call uses no CPU at all and still takes a second of the user's time. A CPU profile shows nothing there; a wall-clock profile shows all of it. For a typical service most of the time is waiting, so wall-clock is usually the more useful of the two.

Allocation and heap are the other pair. Allocation profiling finds code that creates a large number of short-lived objects, which costs garbage collection time but leaks nothing. Heap profiling finds what is still referenced and should not be.

Continuous profiling means it runs in production, all the time. Older profilers were attached during an investigation, by which point the interesting moment had passed and the overhead was too high to leave on. A continuous profiler costs a few percent and is always recording, so the data for last Tuesday's spike is already there.

Profiles are read as a flame graph, where the width of a frame is the time spent in it. The widest frame is where the time went.

19. SLIs and SLOs and SLA

Three words for three different things, and they build on each other.

SLI what we measure
SLO the target we set for it
SLA the promise we make to a customer about it

A worked example, for one endpoint of the payroll API.

First the SLI. It has to be something a user would notice, so it is the share of requests that were both successful and quick:

$$\text{SLI} = \frac{\text{requests that returned a non-5xx response in under 300 ms}}{\text{all valid requests}}$$

Then the SLO, which is a target for that number over a window, and the SLA, which is the contractual version:

	For the payroll API
SLI	Share of requests under 300 ms with a non-5xx response
SLO	99.9% over 30 days, internal
SLA	99.5% over 30 days, or the customer gets a 10% credit

Now one month's real numbers:

Measure	Value
Valid requests	12,000,000
Failed or slower than 300 ms	9,600
SLI achieved	99.92%
SLO target	99.9%, so it was met
Error budget for the month	12,000 requests
Budget used	9,600, which is 80%
Budget remaining	2,400, which is 20%

The error budget is what the SLO leaves over. At 99.9% of 12,000,000 requests, 12,000 of them are allowed to fail. Having used 9,600 of those with ten days to go is the useful signal: the target was met, and there is very little room left to spend on a risky release.

For an availability SLI measured in time rather than requests, the same arithmetic gives a duration:

SLO	Budget per 30 days
99%	About 7 hours
99.9%	About 43 minutes
99.95%	About 22 minutes
99.99%	About 4 minutes

A good SLI measures something the user feels. Request success and latency qualify. CPU does not: a busy machine that serves every request correctly is not an incident.

In Datadog an SLO is built on either a monitor or a ratio of two metric queries, and the product then tracks the budget and how fast it is being spent.

20. Why do we need SLIs, SLOs and an SLA?

Because without a number, "is it reliable enough" is an argument rather than a question with an answer.

	Set by	Audience	Consequence of missing it
SLI	Us	Internal	None. It is a measurement
SLO	Us	Internal	A decision to stop and fix
SLA	A contract	The customer	Money, usually a credit

The SLO is set tighter than the SLA, so that missing the internal target is a warning rather than a bill.

The error budget turns reliability into a decision. Budget remaining means there is room to keep shipping features. Budget spent means the next work is reliability. That replaces the argument about whether a regression is serious enough to act on.

Do not set an SLO of 100 percent. It leaves no budget, which means no release is ever acceptable, and the target gets ignored within a month instead of guiding anything.

The pieces together

Piece	Answers
Metric	How many, how fast
Log	What happened to this one request
Trace	Where the time went
Event	What changed, and when
Dashboard	What is the state of this service

Piece	Answers
Monitor	The rule that decides something is wrong
Alert	The message that rule sends
Agent	What collects and ships the data
DogStatsD	How our own metrics reach the Agent
Integration	Prebuilt collection for a known technology
Tag	What joins all of it together
APM	Which endpoint is slow, and which call inside it
Java Agent	How a JVM service gets traced without a code change
Profiling	Which method inside the slow endpoint
SLI and SLO	Whether it is reliable enough, as a number
Error budget	Whether the next work is a feature or a fix

For example, a release goes out and a deploy event lands on the dashboard. Ten minutes later the error rate metric crosses a monitor's threshold and an alert reaches the on-call channel. The dashboard shows the rise starting at the deploy marker. Filtering traces by `service:payroll version:1.4.2` shows the time going to one downstream call, and the logs for those traces carry the exception. The fix is a rollback to `1.4.2`'s predecessor, and the same dashboard shows the error rate returning to normal.

Tags are the part worth getting right first. Everything above only works because `env`, `service` and `version` mean the same thing in every signal.